inmind.academy
by inmind.ai

Legacy Angular Features
AI For Frontend

SUMMER 2026

inmind.academy
by inmind.ai

Legacy Angular features

inmind.academy
by inmind.ai

NgModules

Legacy: NgModule

- Module in Angular refers to a container that groups similar components, directives, pipes, services... which do a certain task.
- Every Angular app had to use NgModules

Problems

- Too much boilerplate code
- Hard to understand for new developers
- You must jump between files to follow logic
- Small apps still needed modules
- Easy to forget where something is declared

```
@NgModule({
  declarations: [UserComponent],
  imports: [CommonModule],
  providers: [UserService],
  exports: [UserComponent]
})
export class UserModule {}
```

inmind.academy
by inmind.ai

Modern: Standalone Components

What changed?

- Components no longer need NgModules
- Components are standalone
- Each component defines its own dependencies

What is better now?

- No module file needed
- Everything is in one place
- Easier to read and maintain
- Faster onboarding for new developers
- Better tree-shaking and performance

Status: Deprecated direction (still supported)

```
@Component({
  standalone: true,
  selector: 'app-user',
  imports: [CommonModule],
  template: `<p>User works!</p>`
})
export class UserComponent {}
```

inmind.academy
by inmind.ai

State Management & Reactivity

inmind.academy
by inmind.ai

Legacy: RxJS for Local State

How state was handled before?

- Angular depends heavily on RxJS
- Local component state used: Observable, Subject, BehaviorSubject
- Templates used async pipe

```
count$ = new BehaviorSubject(0);

increment() {
  this.count$.next(this.count$.value + 1);
}
```

Problems

- Too much code for simple state
- Hard to understand for beginners
- Manual subscription management
- Easy to cause memory leaks

```
<p>Count: {{ count$ | async }}</p>
```

inmind.academy
by inmind.ai

Modern: Signals

What are Signals?

- Signals are simple reactive variables
- They automatically update the UI
- No subscriptions needed

```
count = signal(0);

increment() {
  this.count.update(v => v + 1);
}
```

```
<p>Count: {{ count() }}</p>
```

What is better now?

- Less code
- No memory leaks
- Very easy to read
- Clear reactive behavior
- Better performance
- Additional features: computed, effect...

Status: RxJS is still important, but not for everything

inmind.academy
by inmind.ai

Component Communication

Legacy: @Input() / @Output()

inmind.academy
by inmind.ai

What are @Input and @Output?

- @Input() receives data from parent
- @Output() sends events to parent
- Based on decorators and events

Problems

- Decorators hide reactivity
- Harder to compose logic
- Does not work well with signals
- Extra boilerplate

TypeScript

```typescript
@Component({
  selector: 'app-child',
  template: `<button (click)="notify()">Click</button>`
})
export class ChildComponent {
  @Input() value!: number;
  @Output() changed = new EventEmitter<number>();

  notify() {
    this.changed.emit(this.value + 1);
  }
}
```

HTML

```html
<app-child
  [value]="count"
  (changed)="count = $event">
</app-child>
```

inmind.academy
by inmind.ai

Modern: Signal Inputs & Outputs

What changed?

- Inputs and outputs are now signals
- No decorators needed

What is better now?

- Inputs are reactive values
- No decorators
- Cleaner and shorter code
- Easy to combine with signals
- Works naturally with computed and effect
- Better performance

Status: Stil supported

TypeScript

```typescript
@Component({
  standalone: true,
  selector: 'app-child',
  template: `<button (click)="changed.emit(value() + 1)">Click</button>`
})
export class ChildComponent {
  value = input<number>();
  changed = output<number>();
}
```

HTML

```html
<app-child
  [value]="count"
  (changed)="count = $event">
</app-child>
```

inmind.academy
by inmind.ai

Change Detection

Legacy: Zone.js

inmind.academy
by inmind.ai

How it worked?

- Angular automatically detects changes using Zone.js
- Whenever something happens, Angular runs change detection on the whole app
- Forces Angular to check everything on every async event

Problems

- Performance overhead on large apps
- Hard to predict when changes happen
- Difficult to debug
- Forces Angular to check everything on every async event
- Hard to integrate with other frameworks or libraries

```
<button (click)="count = count + 1">Increment</button>
<p>{{ count }}</p>
```

inmind.academy
by inmind.ai

Modern: Zoneless Angular & Signals

What changed?

- Angular supports Zoneless mode
- Change detection can now be triggered explicitly
- Signals notify Angular only when needed

What is better now?

- Faster performance (less unnecessary checking)
- Predictable updates
- Easier debugging
- Works better with modern frameworks & libraries
- Fits modern reactive programming style

```
count = signal(0);

increment() {
  this.count.update(v => v + 1);
}
```

HTML

```
<p>{{ count() }}</p>
<button (click)="increment()">Increment</button>
```

inmind.academy
by inmind.ai

Lifecycle Hooks

Legacy: Lifecycle Hooks

inmind.academy
by inmind.ai

Common Lifecycle Hooks

- ngOnInit() → component starts
- ngOnChanges() → input changes
- ngAfterViewInit() → view is ready
- ngOnDestroy() → component is removed

Problems

- Too many methods to remember
- Logic spread across the class
- Easy to forget cleanup
- Hard to compose and reuse logic
- Strongly tied to class-based components

```
export class UserComponent implements OnInit, OnDestroy {
  ngOnInit() {
    this.loadData();
  }

  ngOnDestroy() {
    this.subscription.unsubscribe();
  }
}
```

inmind.academy
by inmind.ai

Modern: Effects & DestroyRef

What changed?

- More focus on reacting to data changes then methods
- effect()
- DestroyRef
- takeUntilDestroyed()

What is better now?

- Less boilerplate
- Logic stays close together
- Automatic cleanup
- Easier to read and test
- Works well with signals

Do not use signals inside effect!

```
count = signal(0);

effect(() => {
  console.log('Count changed:', this.count());
});
```

```
const destroyRef = inject(DestroyRef);

destroyRef.onDestroy(() => {
  console.log('Component destroyed');
});
```

```
this.form.valueChanges
  .pipe(takeUntilDestroyed())
  .subscribe(value => {
    console.log(value);
  });
```

inmind.academy
by inmind.ai

Dependency Injection

Legacy: Constructor Injection

inmind.academy
by inmind.ai

How it worked before?

- Dependencies were injected only in constructors

Problems

- Works only in classes
- Hard to reuse logic
- Constructor becomes large
- Not friendly for functional code
- Logic tied to Angular classes

```
export class UserComponent {
  constructor(private userService: UserService) {}
}
```

inmind.academy
by inmind.ai

Modern: `inject()` Function

What changed?

- Angular introduced the inject() function
- Dependencies can be injected anywhere

What is better now?

- Cleaner code
- No constructor needed
- Enables functional APIs
- Easier to refactor
- Better tree-shaking

```
@Component({...})
export class UserComponent {
  userService = inject(UserService);
}
```

inmind.academy
by inmind.ai

Routing

Legacy: Module-based Routing

inmind.academy
by inmind.ai

How it worked before?

- Routing was defined inside NgModules
- You needed: AppRoutingModule, Feature routing modules
- Lazy loading used module syntax

Problems

- Too many files
- Routing logic spread across modules
- Hard to follow navigation flow
- Lazy loading syntax was verbose
- Strong dependency on NgModules

```
const routes: Routes = [
  {
    path: 'users',
    loadChildren: () =>
      import('./users/users.module')
        .then(m => m.UsersModule)
  }
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule {}
```

inmind.academy
by inmind.ai

Modern: Standalone Routing

What changed?

- Routes are standalone
- No routing modules needed
- Providers can live on routes

What is better now?

- Fewer files
- Easier to read
- Faster lazy loading
- Routes match the app structure
- Works perfectly with standalone components

```typescript
export const routes: Routes = [
  {
    path: 'users',
    loadComponent: () =>
      import('./users/users.component')
        .then(c => c.UsersComponent)
  }
];
```

TypeScript

```typescript
bootstrapApplication(AppComponent, {
  providers: [provideRouter(routes)]
});
```

```typescript
{
  path: 'admin',
  loadComponent: () =>
    import('./admin/admin.component').then(c => c.AdminComponent),
  providers: [AdminService]
}
```

Extra Improvement: Functional Guards & Resolvers

inmind.academy
by inmind.ai

Before

- Class-based guards
- Interfaces like CanActivate

```
@Injectable({ providedIn: 'root' })
export class AuthGuard implements CanActivate {
  canActivate() {
    return isLoggedIn();
  }
}
```

sed like:

TypeScript

```
{
  path: 'dashboard',
  canActivate: [AuthGuard],
  loadComponent: () =>
    import('./dashboard/dashboard.component').then(c => c.DashboardComponent)
}
```

What is better now?

- No classes
- Less boilerplate
- Easier to test
- Uses inject()

```
export const authGuard = () => {
  const auth = inject(AuthService);
  return auth.isLoggedIn();
};
```

inmind.academy
by inmind.ai

Forms

Legacy: Angular Forms

inmind.academy
by inmind.ai

How it worked before?

- Classic Reactive Forms (FormGroup, FormControl)
- Heavy use of classes and lifecycle hooks

```
this.form = new FormGroup({
  name: new FormControl('', Validators.required),
  email: new FormControl('', Validators.required),
});
```

Problems

- Too much boilerplate
- Logic is spread everywhere
- You subscribe manually
- forms harder to read, test, and maintain

```
ngOnInit() {
  this.form.valueChanges.subscribe(value => {
    this.isValid = this.form.valid;
  });
}

ngOnDestroy() {
  this.subscription.unsubscribe();
}
```

inmind.academy
by inmind.ai

Modern: Signals + Typed Forms

What changed?

- Typed Reactive Forms
- Signals for form state

What is better now?

- Less boilerplate
- Fully typed (no null issues)
- Clear, reactive logic
- No manual subscriptions
- Easier to test
- Showing validation cleanly

```
form = this.fb.nonNullable.group({
  name: ['', Validators.required],
  email: ['', Validators.required],
});
```

```
formValue = toSignal(this.form.valueChanges, {
  initialValue: this.form.value,
});

isFormValid = computed(() => this.form.valid);
```

```
<button [disabled]="!isFormValid()">Save</button>
```

inmind.academy
by inmind.ai

Structural Directives

Legacy: Structural Directives

inmind.academy
by inmind.ai

How it worked before?

- *ngIf
- *ngFor
- *ngSwitch

```
<div *ngIf="user">
  <div *ngFor="let item of user.items">
    <span *ngIf="item.active">{{ item.name }}</span>
  </div>
</div>
```

Problems

- The * syntax is confusing and not easy
- Uses hidden <ng-template>
- Hard to follow logic
- No clear “else” structure

```
<div *ngIf="isLoading; else content">
  Loading...
</div>

<ng-template #content>
  Data loaded
</ng-template>
```

Modern: New Control Flow Syntax

What changed?

- @if
- @for
- @switch

What is better now?

- Looks like real code
- Less magic
- Easier for beginners
- Better readability
- Easier maintenance

```
@if (user) {
  @for (item of user.items) {
    @if (item.active) {
      <span>{{ item.name }}</span>
    }
  }
}
```

inmind.academy
by inmind.ai

Build & Tooling

inmind.academy
by inmind.ai

Legacy:Webpack

How it worked before?

- Based on Webpack
- Very configurable
- But also very heavy

```
module.exports = {
  module: {
    rules: [
      { test: /\.ts$/, use: 'ts-loader' }
    ]
  }
};
```

Problems

- Slow build and serve times
- Webpack config is big and hard to understand
- Bad Tree-shaking because it dependes on modules

Most developers didn’t touch this

inmind.academy
by inmind.ai

Modern: esbuild and Vite

What changed?

- Uses esbuild

What is better now?

- Much faster and simpler
- Changes appear almost immediately
- Better tree-shaking
    - uses functional APIs and
    - Removes unused features automatically

inmind.academy
by inmind.ai

AI Tools

inmind.academy
by inmind.ai

UI Generators

inmind.academy
by inmind.ai

Manus

What it is?

- Complete a large task
- Generates full UI screens
- Focuses on clean layouts and design

Why It’s useful?

- Saves design + coding time
- Good for starting projects quickly
- Helpful for non-designers

inmind.academy
by inmind.ai

Lovable

What it is?

- Quickly build a working web app
- You describe an idea
- It builds UI + basic logic
- Very beginner-friendly

Why It’s useful?

- Very good for prototypes
- Helps frontend + product thinking
- Less technical than other tools

inmind.academy
by inmind.ai

Replit

What it is?

- Build and edit a real code project with full control
- An online code editor in the browser
- With AI that write and edit frontend code, fix bugs, and explain code

Why It’s useful?

- No setup needed
- AI helps while coding
- Great for learning + experimenting

inmind.academy
by inmind.ai

Google Stitch

What it is?

- Build UI/UX design ideas and prototypes
- Converts UI designs into frontend code
- Focuses on design → code

Why It’s useful?

- Faster move from design to implementation
- Cleaner initial frontend structure

inmind.academy
by inmind.ai

Summary

- **Need design ideas?** → Stitch 🎨
- **Need a quick app?** → Lovable 🚀
- **Need to code and learn professionally?** → Replit 💻
- **Need an AI worker to handle a big complex task?** → Manus 🤖

inmind.academy
by inmind.ai

UI Agents

inmind.academy
by inmind.ai

Types of AI Agents

- **Ask Agent**: Answers questions and explains concepts to help you learn or debug quickly
- **Plan Agent:** Breaks a feature or problem into clear, logical steps before coding
- **Act / Build Agent:** Writes and executes code to build features fast
- **Review Agent**: Analyzes your code and suggests improvements for quality and best practices

inmind.academy
by inmind.ai

Cursor

inmind.academy
by inmind.ai

Cursor

What it is?

- A code editor like VS Code
- With deep AI integration

Skill files

- Cursor has skill files which are rules you give to the AI to specify:
  - How to write code
  - What patterns to follow

Why this is powerful?

- Consistent code
- Team standards
- Less review effort

inmind.academy
by inmind.ai

The big picture

- **AI is like a Junior Developer:** Helps write code and fix small issues, but needs review
- **AI for Prototyping:** Try UI ideas fast and discard them if they are not good
- **Limits of AI:** Sometimes wrong and doesn’t know your business rules
- **Future:** Less boring code, more focus on UX and smart decisions

inmind.academy
by inmind.ai

THANK YOU

Questions?

ADDRESS

Mkalles, main road, bld. 757, Lebanon

CONTACT

Phone +961 70 478 758
Email inmind.academy@inmind.ai
Website www.inmind.academy

SOCIAL MEDIA